



Figure 4: Placing *google-services.json* in Android Studio SDK

with the created account. This can be done by clicking on 'Add Firebase to your Android app' as shown in Figure 2, that will pop up a window like the one seen in Figure 3, prompting you to enter the package name of your Android project. Once added, click on 'Add App'.

- The website then generates a *config* file named *google-services.json* using the package name provided in Step 4. This file needs to be placed in your Android Studio app's root directory. All these instructions are displayed on the Firebase site (Figure 4).
- Google-services.json* is needed during the compile time of the Android app. It indicates which Firebase URL/account the app needs to connect with to store and retrieve data. In this case, it happens to be the account you have just created using Google credentials.

With this, the environment needed for using Firebase has been set up. We are now ready to use Firebase APIs to push and pull data from the mobile device.

A simple application for pushing data

Let me now illustrate a sample Android application that writes weather updates to Firebase's cloud database which, in turn, notifies other clients registered to this account. Let us take a couple of cities, say Bengaluru and Delhi, and their corresponding temperature in Celsius, humidity in percentage and atmospheric pressure in millibars. Then let's try to update the Firebase database from the app.

Firebase APIs will try to write the Java object's values to the cloud's database. So, we first need to create a class, say *Weather* as shown below, then create objects of this class representing different cities and populate the objects with temperature, humidity and atmospheric pressure values. Here, the classes for serialisation expect to contain a default blank constructor.

```
public class Weather {
    private String city;
    private int temperature;
    private int humidity;
    private int pressure;
```

```
public Weather() {
}
public void setCity(String city) {
    this.city = city;
}
public void setTemperature(int temperature) {
    this.temperature = temperature;
}
public void setHumidity(int humidity) {
    this.humidity = humidity;
}
public void setPressure(int pressure) {
    this.pressure = pressure;
}
public String getCity() {
    return city;
}
public int getTemperature() {
    return temperature;
}
public int getHumidity() {
    return humidity;
}
public int getPressure() {
    return pressure;
}
}
```

With this class, one can create objects representing Bengaluru and Delhi, and use setter methods to set temperature, humidity and pressure. Once the values are set, they can be written to the database like in the following code snippet:

```
DatabaseReference rootRef = FirebaseDatabase.getInstance().
    getReference();
Weather bangalore = new Weather();
bangalore.setCity("Bangalore");
bangalore.setTemperature(24);
bangalore.setHumidity(83);
bangalore.setPressure(1013);
rootRef.child("Bangalore").setValue(bangalore);

Weather delhi = new Weather();
delhi.setCity("Delhi");
delhi.setTemperature(27);
delhi.setHumidity(94);
delhi.setPressure(1003);
rootRef.child("Delhi").setValue(delhi);
```

FirebaseDatabase.getInstance().getReference() returns a reference to the real-time database you got on creating your Firebase account. The app gets compiled with details of the database URL from *google-services.json* that you had downloaded and added to your project. When the app runs